COMPLETE ENGINEERING SCOPE

Agentic SDD Platform

Full technical scope and step-by-step implementation plan — covering both greenfield feature delivery and feature delivery into existing, already-running repositories.

Prepared by
Pranav

Scope
Full production design (not PoC-limited)

Document type
Engineering Scope & Implementation Plan

Status
Working reference document

Table of Contents

1. Vision & Scope 1

2. Complete System Architecture 2

3. Technology Stack 4

4. Repository Layout 5

5. Workflow Status Model 6

6. The Context Agent — Repository Indexing Pipeline 7

7. Traceability Knowledge Graph 9

8. Case A — Building a Completely New Feature 10

9. Case B — Implementing a Feature in an Existing Project 11

10. Full Step-by-Step Build Sequence 13

11. Governance Model 15

12. Success Metrics & Key Risks 16

13. Appendix — Key Design Decisions & Rationale 17

1. Vision & Scope

The Agentic SDD Platform governs how a business requirement becomes production code. It orchestrates AI agents across Confluence, Jira, Git, Datadog, and Claude Code so that requirements, designs, tasks, and code stay explicitly linked — while every product, architecture, and release decision continues to require human approval.

This document is the full production scope — not the cut-down version built for a one-week proof of concept. It covers every component discussed: the Orchestrator, all six agents, the Context Agent (repo indexing pipeline), the traceability knowledge graph, and — the focus of this version — exactly how the system behaves differently when a feature lands in a **brand-new codebase** versus an **existing, already-indexed repository**.

✓ In scope

- ✓ Orchestrator with full 18-status workflow state machine
- ✓ Requirement, Design, Task, Implementation, Review/Validation, and Impact Agents
- ✓ Context Agent: chunking, embeddings, nested CLAUDE.md, mind maps, dependency tables
- ✓ Traceability knowledge graph (Neo4j) across every linked system
- ✓ sdd-mcp-server for Claude Code / VS Code integration
- ✓ Both greenfield and brownfield feature workflows

✕ Explicitly not in scope here

- ✕ Fully autonomous implementation with no PR-stage human gate
- ✕ Replacing Confluence, Jira, Git, VS Code, or Claude Code
- ✕ A specific UI visual design (covered separately)
- ✕ Multi-cloud / multi-tenant SaaS packaging of the platform itself

2. Complete System Architecture

The Orchestrator is a backend control-plane service. It owns workflow state and integrations; every agent reasons and drafts, but never independently controls approvals or state transitions.

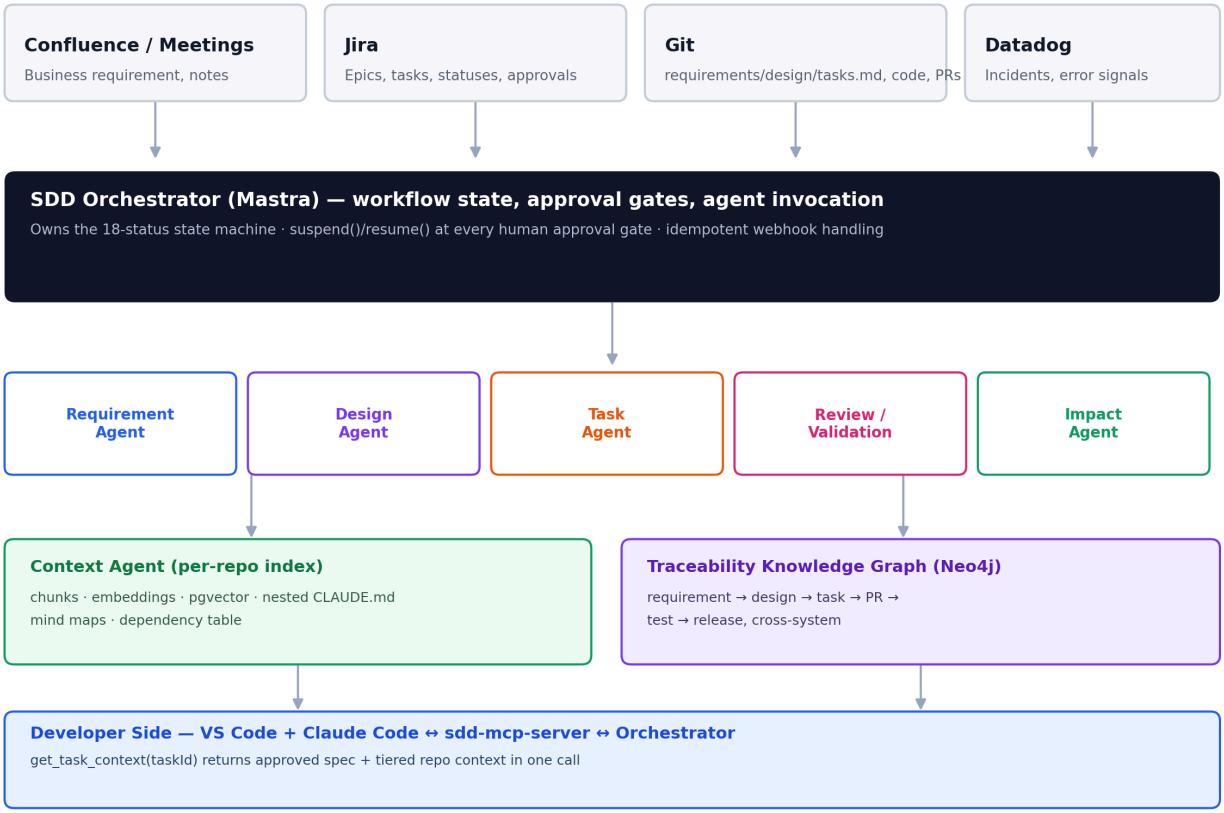


Figure 1 — Full system architecture: systems of record, Orchestrator, agents, Context Agent, knowledge graph, developer side.

Component responsibilities

Component	Responsibility
Orchestrator	Manages workflow state and stage transitions; reacts to Jira/Git/user/agent events; invokes the correct agent with correctly versioned context; enforces human approval gates; persists agent runs, questions, approvals, and artifact references; ensures idempotent webhook/event handling.
Requirement Agent / Critic	Reads the source requirement (Confluence, meeting notes, or a Datadog-triggered incident); drafts `requirements.md`; raises clarification questions; revises incrementally until approved.
Design Agent / Critic	Reads approved requirements plus repository context (from the Context Agent); drafts `design.md`; raises design questions; revises after architecture review.
Task Agent	Decomposes an approved design into `tasks.md` and linked Jira development tasks, with dependencies mapped.
Context Agent	Indexes a repository once (and incrementally thereafter): chunking, embeddings, nested CLAUDE.md, mind maps, dependency table. Serves tiered context to every other agent and to developers directly.
Impact Agent	When an approved requirement or design changes, walks the dependency graph to flag which downstream tasks, PRs, or tests are now potentially stale.

Component	Responsibility
Review / Validation Agent	On PR creation, diffs the PR against the requirement/design/task it claims to implement; checks test coverage against acceptance criteria.
sdd-mcp-server	Exposes `get_task_context(taskId)` and related tools so Claude Code retrieves approved spec + tiered repo context in one call, instead of the developer reconstructing it manually.

3. Technology Stack

Layer	Technology	Why
Orchestrator + all agents	TypeScript + Mastra	Mastra's `suspend()` / `resume()` workflow primitive maps directly onto human approval gates; one language across the whole backend and frontend.
Shared schemas	Zod, in a standalone `sdd-contracts` package	Every repo (Orchestrator + each agent) pins a version — prevents schema drift across a split-by-agent repo layout.
Database	PostgreSQL + pgvector	One database serves both the relational workflow state and the embeddings index — no second database technology needed for vector search.
Async / parallel work	Redis + BullMQ	Backs the Phase 3+ parallel build-fix-validate loop; lighter-weight than Kafka/SQS at this scale.
Code chunking	tree-sitter	Language-aware chunking at function/class/route granularity — free, open source (MIT).
Code embeddings	Voyage AI — `voyage-code-3`	Anthropic's recommended embedding partner, tuned for code. 200M free tokens per account, then $0.18/million tokens.
Traceability graph	Neo4j Community Edition	Free, open source (GPLv3); Cypher traversal is a far better fit than recursive SQL for arbitrary-hop "what does this affect" queries.
Developer integration	MCP TypeScript SDK	Anthropic's primary MCP SDK; this is the component developers touch directly, so it should be built on the most mature tooling available.
LLM calls	Anthropic API (Claude)	All agent reasoning and generation.
Frontend	React + TypeScript + Vite + Tailwind	Dashboard, review workspace, traceability views.
Auth	Enterprise SSO (OIDC) + RBAC	Least-privilege access to Jira, Confluence, Git, and the LLM gateway.

Why not embeddings-based RAG everywhere, by default?
Anthropic's own Claude Code team built and then deliberately dropped a vector-DB RAG approach for general code search, finding that agentic search (grep/glob/read, grounded by well-maintained CLAUDE.md files) works better for most repositories — chunking fragments context, embeddings are fuzzy when the exact symbol is already known, and an index is a snapshot that goes stale the moment code changes. The Context Agent in this plan exists specifically because this platform's repositories are large/monorepo-scale, which is exactly the regime where that tradeoff flips back in favor of embeddings. Smaller utility repos should stay on CLAUDE.md + native agentic search alone.

4. Repository Layout

Split by agent for independent scaling and ownership, anchored by one shared contracts package so nothing drifts.

Repo	Purpose
`sdd-contracts`	Shared Zod schemas + the status enum/state machine. Zero dependencies on anything else. Every other repo pins a version.
`sdd-infra`	docker-compose wiring Postgres, Redis, and (later) Neo4j together for local development.
`sdd-orchestrator`	Mastra workflow, state machine, DB, webhook receivers, Jira/Confluence/Git/Datadog adapters.
`sdd-requirement-agent`	Requirement Agent + Critic.
`sdd-design-agent`	Design Agent + Critic.
`sdd-task-agent`	Task Agent.
`sdd-context-agent`	Repo profiler, tree-sitter chunker, embeddings pipeline, nested CLAUDE.md generator, mind-map generator, dependency table, query API.
`sdd-review-validation-agent`	Review Agent + Validation Agent.
`sdd-knowledge-graph`	Neo4j sync worker + Cypher queries backing the Impact Agent and traceability views.
`sdd-mcp-server`	Claude Code / VS Code integration — `get_task_context(taskId)` and related tools.
`sdd-ui`	React frontend — dashboard, review workspace, traceability view.

5. Workflow Status Model

The Orchestrator drives every transition through an explicit Jira-style status model, modeled as Mastra workflow steps with `suspend()` / `resume()` at every human checkpoint. Re-analysis is triggered only by defined status changes — never by every comment — to avoid noisy, unstable artifact churn.

Status	Meaning
BACKLOG → READY_FOR_SPEC	Requirement exists; Product confirms it is ready for analysis.
REQUIREMENT_GENERATING	Requirement Agent is running (suspended workflow step).
REQUIREMENT_REVIEW → REQUIREMENT_REVIEWED	Clarification questions open; Product answers; Agent re-runs.
SPEC_REVIEW → SPEC_APPROVED	No unresolved gaps; Product gives final approval.
DESIGN_GENERATING	Design Agent running, using approved requirements + Context Agent output.
DESIGN_REVIEW → DESIGN_REVISION_REQUESTED / DESIGN_APPROVED	Architecture reviews; either revise or approve.
TASKS_GENERATING → READY_FOR_DEVELOPMENT	Task Agent decomposes design; Jira tasks created.
IN_DEVELOPMENT → READY_FOR_REVIEW	Developer (or the autonomous loop, Phase 3+) implements; PR opened.
QA_VALIDATION → READY_FOR_RELEASE → DONE	Review/Validation Agent + human review + CI/QA; release.

6. The Context Agent — Repository Indexing Pipeline

This is what lets the Requirement and Design Agents reason about an existing codebase without reading it end to end, and what keeps token cost bounded as the repository estate grows.

6.1 When to use the full pipeline vs. CLAUDE.md alone

Repo profile	Approach	Cost
Monorepo-scale (1000+ files) or queried repeatedly across many tasks	Full pipeline: chunking + embeddings + nested CLAUDE.md + mind map + dependency table	tree-sitter/pgvector free; Voyage embeddings pay-per-token past 200M free tokens
Small utility repo (<200 files)	Nested CLAUDE.md + mind map only; rely on Claude Code's native agentic (grep/glob) search	Free

A Repo Profiler job classifies every repo by file count/LOC and keeps this decision current — most of this platform's actual repositories fall into the first row, so the full pipeline is the default build, not an opt-in exception.

6.2 Pipeline stages

1. **Chunking** — tree-sitter parses each file into function/class/route-handler-level chunks (not naive line splitting), tagged with file path, symbol name, and commit SHA.
2. **Embeddings** — each chunk is embedded with `voyage-code-3` and stored in pgvector alongside its metadata.
3. **Dependency table** — a Postgres table of file-to-file imports, plus package/workspace-manifest relationships for monorepos. One or two hops of this answer "what calls this" without graph-DB machinery.
4. **Nested CLAUDE.md generation** — module boundaries are detected (directories with their own manifest file, 8+ source files, or conventional names like `services/`, `routes/`); each gets its own CLAUDE.md built from only its own chunks, plus a root CLAUDE.md indexing all modules. Written as `CLAUDE.md.generated.md` for human review — never silently overwritten.
5. **Mind maps** — a Mermaid diagram + prose summary per repo (internal module structure), plus one system-level mind map linking repos via detected cross-repo references (shared service names, docker-compose hostnames, shared package imports).
6. **Incremental re-indexing** — on every merge to main, diff the commit and re-chunk/re-embed only the changed files, not the whole repo.

6.3 Query-time context resolution

When any agent or developer needs context for a task, the Context Resolver:

1. Embeds the task/feature description and runs a pgvector similarity search for the top-K relevant chunks.
2. Expands one hop via the dependency table to pull in direct callers/callees.
3. Attaches the relevant nested CLAUDE.md section(s) for the touched module(s).
4. Attaches the relevant mind-map excerpt — not the whole repo's, just the touched module(s).
5. Returns a single, size-capped context bundle.

Cost sanity check:
A 1,000-file repo at roughly 1,500 tokens/file chunked is about 1.5M tokens to embed once. Across even 80 such repos, a full first-time pass across the estate is roughly 120M tokens — inside `voyage-code-3` 's 200M free-token allowance. After the first pass, only the diff on each merge gets re-embedded, which is small. Validate against real average file size before quoting this anywhere, but it is not a large number.

7. Traceability Knowledge Graph

A separate concern from the Context Agent: this graph is shallow, cross-system, and about governance — "what does this connect to" — rather than deep single-repo semantic search.

Node types	Edge types
Initiative, RequirementVersion, DesignVersion, TaskVersion, TestPlanVersion, JiraIssue, ConfluencePage, GitCommit, PullRequest, Incident, Person	`HAS_VERSION`, `DERIVED_FROM`, `TRACKS`, `IMPLEMENTS`, `TESTED_BY`, `TRIGGERED`, `APPROVED`

Postgres remains the system of record for all approvals and state transitions (ACID guarantees the graph store doesn't need to provide). The graph is a derived, read-optimized layer kept in sync one-directionally via domain events (`ArtifactVersionCreated` , `ApprovalRecorded` , etc.) over the same Redis/BullMQ queue used elsewhere — never a dual write into both stores in the same transaction.

This is what powers the Impact Agent ("what breaks if I change this requirement") and gives Product/Architecture/QA/Ops one visual, traversable view of a requirement's full lifecycle.

8. Case A — Building a Completely New Feature

"New feature" here means the target repository either doesn't exist yet or has no meaningful history to mine — there is nothing for the Context Agent to index, so the Requirement and Design Agents work from the source requirement and general engineering standards alone.



Figure 2 — Case A workflow: no existing-repo context step; agents work from requirement + standards only.

#	Stage	What happens
1	Create Initiative	Product creates the initiative in Jira with the business requirement and acceptance criteria. No repository is linked yet, or the linked repository is new/empty.
2	Requirement Agent	Drafts `requirements.md` from the source requirement alone, plus organizational requirement-writing standards (no repo context to pull in). Raises clarification questions.
3	Clarify & Approve	Product answers; Requirement Agent revises; Product gives final approval.
4	Design Agent	Drafts `design.md` from approved requirements plus general architecture standards/ADRs — there is no existing codebase to ground the design against, so the Design Agent proposes the structure rather than fitting into one.
5	Review & Approve	Architecture reviews the proposed design via Jira/Git PR; approves or requests changes.
6	Task Agent	Decomposes the approved design into `tasks.md` and Jira development tasks.
7	Implement	Developer implements in VS Code + Claude Code. Since the repo is new, `get_task_context` returns only the approved spec — there is no prior CLAUDE.md or embeddings index to attach yet.
8	Review, Test, Release	AI + human review, CI/QA, release. **Important follow-on:** once this repository has real code and history, it should be registered with the Context Agent and indexed — so the next feature into this same repo runs as Case B, not Case A.

Key implication:
Case A is actually the simpler path technically, but the Design Agent's output carries more risk — it's proposing architecture rather than fitting into constraints a codebase already enforces. Architecture review in step 5 deserves more scrutiny here than in Case B.

9. Case B — Implementing a Feature in an Existing Project

This is the common case given the current repository estate: the target repository already exists, already has conventions, and the whole point of the Context Agent is to make the Requirement and Design Agents aware of them instead of guessing or re-reading the entire repository on every task.

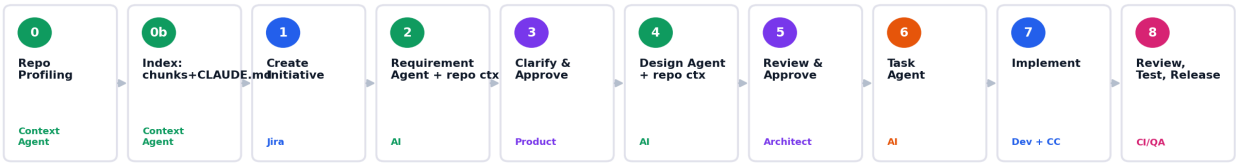


Figure 3 — Case B workflow: an indexing prerequisite (steps 0/0b) feeds real repo context into both the Requirement and Design Agents.

#	Stage	What happens
0	Repo Profiling	Prerequisite, done once per repo (not per feature): the Repo Profiler classifies the repo (file count/LOC) to decide full pipeline vs. CLAUDE.md-only, per the table in Section 6.1.
0b	Index	Prerequisite: tree-sitter chunking, Voyage embeddings into pgvector, dependency table, nested CLAUDE.md, mind map. Re-run incrementally on every merge thereafter — this is standing infrastructure, not re-done per feature.
1	Create Initiative	Product creates the initiative in Jira, linked to the existing, already-indexed repository (or repositories, if the feature spans more than one).
2	Requirement Agent + repo context	The Orchestrator calls the Context Resolver *before* invoking the Requirement Agent — pulling relevant chunks, the touched module's CLAUDE.md, and the relevant mind-map excerpt — so the agent drafts `requirements.md` already aware of what exists, not just what was asked for.
3	Clarify & Approve	Same loop as Case A, but questions are frequently sharper — e.g. the agent can ask "this conflicts with how `billing-service` currently handles retries, which behavior should win?" because it has that context.
4	Design Agent + repo context	Drafts `design.md` grounded in the actual architecture, existing patterns, and the dependency table — proposing how the feature fits into what's there, not a structure in a vacuum.
5	Review & Approve	Architecture review is typically faster here — the design already respects existing constraints, so review is about fit and risk, not first-principles soundness.
6	Task Agent	Same as Case A.
7	Implement	`get_task_context(taskId)` returns the approved spec **plus** the tiered repo context bundle — real chunks, real CLAUDE.md, real dependency edges. This is the step that most reduces token cost and context drift versus a developer manually re-deriving context each time.
8	Review, Test, Release	Same as Case A; the Impact Agent also checks whether this change makes any other indexed dependent repo's context stale.

Key implication:
Steps 0 and 0b are standing infrastructure maintained continuously (incremental re-index on every merge), not a per-feature cost. Once a repo is indexed, every subsequent feature into it runs the cheaper, better-grounded Case B path — which is why indexing the existing repository estate early is worth prioritizing over indexing repos reactively, feature by feature.

10. Full Step-by-Step Build Sequence

Ordered so each step produces something demonstrable before the next begins. This is the complete production sequence — the earlier one-week PoC plan is a deliberately-cut subset of this for a single demo, not a different plan.

Phase 0 — Foundation

1. `sdd-contracts`: Zod schemas for all core entities, the 18-status state machine, versioned via Changesets.
2. `sdd-infra`: docker-compose for Postgres (with pgvector enabled) + Redis.
3. `sdd-orchestrator` skeleton: Mastra workflow modeling every status as a step, `suspend()` / `resume()` at each approval gate, no real agent calls yet — just the state machine and a test walking every legal/illegal transition.
4. Thin, read-first integration adapters: fetch a Confluence page, create a Jira Epic, read/write to `/specs/<initiative-id>/` in Git.
5. Idempotent webhook receivers for Jira + Git (dedupe by event ID) — build this now, not retrofitted later.

Phase 1 — Requirement Loop

1. LLM gateway wrapper around the Anthropic API — one function, structured output schema.
2. `sdd-requirement-agent`: Confluence content in → `requirements.md` (v1) + questions out.
3. Answer path: questions posted as Jira comments (or, for faster iteration, the SDD UI directly); replies feed back for re-analysis.
4. Wire `REQUIREMENT_REVIEW → REQUIREMENT_REVIEWED → SPEC_REVIEW → SPEC_APPROVED`.

Phase 2 — Context Agent + Design Loop

Pulled forward relative to earlier plans: given most repositories here are monorepo-scale, the Context Agent needs to exist before the Design Agent can produce grounded output, not after.

1. `sdd-context-agent`: Repo Profiler, tree-sitter chunker, `voyage-code-3` embeddings into pgvector, dependency table.
2. Nested CLAUDE.md generator (module detection → per-module file → root index file), written as `.generated.md` for human review.
3. Mind-map generator: per-repo + cross-repo (via detected shared references).
4. Context Resolver + query API, consumed first by the Design Agent.
5. `sdd-design-agent`: approved requirements + Context Agent output → `design.md` + design questions → architecture review loop → `DESIGN_APPROVED`.

Phase 3 — Tasks & Developer Integration

1. `sdd-task-agent`: approved design → `tasks.md` + linked Jira sub-tasks.
2. `sdd-mcp-server`: `get_task_context(taskId)` — approved spec + Context Resolver bundle in one call, wired into Claude Code.

Phase 4 — Review, Validation & Traceability

1. `sdd-review-validation-agent`: on PR creation, diff against the linked requirement/design/task; check acceptance criteria and test coverage.
2. Continuous rule improvement: when a human edits an agent-generated PR, capture the diff as feedback and fold it into the repo's CLAUDE.md/rule file, so accuracy compounds instead of resetting each run.

3. `sdd-knowledge-graph`: Neo4j + event-driven sync worker from Postgres; backs the Impact Agent and a visual traceability view in the SDD UI.

Phase 5+ — Increased Autonomy (opt-in, per the governance model)

1. Autonomous build-fix-validate loop: Implementation Agent runs isolated per-task branches/sandboxes, parallelized across independent tasks via BullMQ — human approval stays exactly at PR review/merge, never inside the loop.
2. Datadog ingestion as a requirement source — an incident or error pattern can spawn or inform an initiative.
3. Generalize beyond code repos: let the knowledge graph pull in meeting notes and other linked platforms as they're added.

11. Governance Model

Governance is enforced structurally by the Orchestrator's state machine, not left to agent discretion or convention.

Action	Who decides
Approve requirements	Product — configured approver, never an agent
Approve design	Architecture — configured approver, never an agent
Generate tasks	System, automatically, but only after both approvals above are recorded
Merge PR	Existing Git permissions — no AI auto-approval, in any phase
Release	Existing release governance and CI/CD permissions, unchanged

As autonomy increases (Phase 5+), gates must scale with it, not relax.
The more a system can act on its own, the larger the blast radius of a misconfiguration. The parallelized autonomous loop increases what agents can *attempt*; it must not reduce what still requires a human decision before it ships.

12. Success Metrics & Key Risks

Success measures

- % of ambiguities resolved before development starts
- Cycle time from requirement to approved design
- % of requirements mapped to tasks and test evidence
- % of dev tasks started with approved SDD context vs. manual reconstruction
- Token/cost per task, Case A vs. Case B (validates the Context Agent investment)
- Reduction in late requirement/design changes post-release

Key risks & mitigations

- Weak source requirement → Requirement Critic + mandatory approval
- Design Agent proposes architecture ungrounded in reality (Case A) → stricter architecture review for new repos
- Embeddings index goes stale → incremental re-index on every merge, never a static snapshot
- Noisy re-runs → triggered only by explicit status changes
- Governance risk as autonomy increases → gates scale with capability, per Section 11

13. Appendix — Key Design Decisions & Rationale

Decision	Rationale
TypeScript + Mastra over Python + FastAPI	`suspend()` / `resume()` maps directly onto approval gates; the MCP TypeScript SDK is Anthropic's primary one; one language across the whole stack including the frontend.
Split-by-agent repo layout	Scales independently per team/agent; risk of schema drift is handled by a single shared `sdd-contracts` package every repo pins, not by merging repos back together.
Full Context Agent pipeline as default, not opt-in	Most repositories in this estate are already monorepo-scale (1000+ files) or queried repeatedly — exactly the regime where Anthropic's own data shows embeddings outperform pure agentic search, rather than the general case where it doesn't.
Nested CLAUDE.md over one root file	At this scale, one root file is either too vague to be useful or too long to load every session; Claude Code loads nested files automatically when working in that directory.
Context Agent (pgvector) kept separate from the Knowledge Graph (Neo4j)	Different jobs: semantic relevance within one repo (vector similarity) versus shallow cross-system governance traversal (graph). Conflating them would force one database technology to do a job it isn't built for.
Human approval only at PR/merge for the autonomous loop, never inside it	Matches the governance-first premise of the whole platform: more autonomy must come with gates that scale, not gates that get removed for convenience.